

DDA4300 Final Project Report

Project 7: Online Linear Programming and Resource Allocation

Frederick Khasanto (122040014), Stefan Susanto (122040041)

April 30, 2025

Contents	12 Conclusion	7
1 Introduction	2	
2 Problem Setup	2	
3 Question 1: One-Time Online Learning Sensitivity	2	
3.1 Overview	2	
3.2 Results and Discussion	2	
4 Question 2: Dynamic Price Updates	3	
4.1 Overview	3	
4.2 Results and Discussion	3	
5 Question 3: Convex Model and KKT Conditions	3	
6 Question 4: Partial Convex Program	4	
6.1 Overview	4	
6.2 Results and Discussion	4	
7 Question 5: Dynamic Convex Pricing	5	
7.1 Overview	5	
7.2 Results and Discussion	5	
8 Question 6: Action-History Learning Algorithm	5	
8.1 Overview	5	
8.2 Results and Discussion	5	
9 Question 7: Convex Formulation and Equivalence	6	
10 Question 8: Stochastic Gradient Descent Algorithm	6	
10.1 Overview	6	
10.2 Results and Discussion	6	
11 Question 9: Violation of i.i.d. Assumptions	7	

1 Introduction

In many real-world applications such as online advertising, resource scheduling, and cloud computing, decisions must be made in an online fashion—before the full set of input data is available. This course project explores such a setting through the lens of *Online Linear Programming* (OLP) for resource allocation.

The project focuses on comparing multiple strategies for solving an LP problem where allocation decisions must be made sequentially. We investigate both linear and convex formulations, implement online algorithms that rely on dual price estimations, and compare them to the classical offline optimal solution. Our goal is to understand the trade-offs in performance, robustness, and computational cost among the different algorithms, particularly when the problem data is stochastic or adversarial.

Through simulation-based experiments, we evaluate the sensitivity of revenue performance to various hyperparameters, the convergence of dual prices, and the impact of relaxing the i.i.d. assumption. We also explore modern optimization techniques such as stochastic gradient descent to address computational bottlenecks in large-scale LPs.

2 Problem Setup

We consider the following linear programming formulation for resource allocation:

$$\begin{aligned} & \text{maximize}_{\mathbf{x}} && \sum_{j=1}^n \pi_j x_j \\ & \text{subject to} && \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad \forall i = 1, \dots, m, \\ & && 0 \leq x_j \leq 1, \quad \forall j = 1, \dots, n \end{aligned}$$

where:

- x_j is the decision variable indicating the allocation for bidder j .
- π_j is the bid (or gain) associated with bidder j .
- $a_{ij} \in \{0, 1\}$ is the binary indicator of whether bidder j requires resource i .
- b_i is the available quantity of resource i , fixed at $b_i = 1000$ in our experiments.

We simulate a stream of $n = 10,000$ bidders. For each bidder k , we:

- Randomly generate $a_k \in \{0, 1\}^m$,
- Fix a “ground truth” price vector $\bar{p} > 0$,
- Define $\pi_k = \bar{p}^\top a_k + \text{randn}(0, 0.2)$, where the Gaussian noise models imperfect valuation.

Throughout the project, we compare the performance of online algorithms with the optimal solution obtained by solving the full LP offline using all n bidders.

3 Question 1: One-Time Online Learning Sensitivity

3.1 Overview

We begin our investigation by implementing the one-time online learning algorithm, which uses the first k bidders to estimate dual prices and then applies those fixed prices to make decisions for the remaining $n - k$ bidders. This approach was introduced and analyzed in [3].

We fix $n = 10,000$ and $m = 10$, with $b_i = 1000$ for all i . We compare three values of k : 50, 100, and 200, and evaluate the performance ratio:

$$\text{Revenue Ratio} = \frac{\text{Online Revenue}}{\text{Offline Optimal Revenue}}$$

for each value of k .

3.2 Results and Discussion

When k is too small, the limited sample size may not adequately capture the structure of the bidding environment. Consequently, the estimated prices $\bar{y}_k$ may not reflect the true marginal value of the resources, leading to inefficient allocation decisions—such as accepting low-value bids or prematurely exhausting certain resources. However, the algorithm benefits from having nearly the entire bidder population available for allocation, which may partially compensate for the poor price estimates.

In contrast, a large value of k yields more reliable dual prices that align better with the true underlying resource values. Yet, this comes at the cost of reducing the number of bidders remaining to act upon those estimates. In such cases, even if the dual prices are close to optimal, the limited decision horizon diminishes the

algorithm's capacity to fully capitalize on that information.

k	Online Revenue	Ratio $\rho(k)$
50	5063.80	0.876
100	4686.01	0.811
200	4668.72	0.808

Table 1: Online Revenue and Ratio $\rho(k)$ for Different Values of k

Our results in 1 confirm this tradeoff that while increasing k improves pricing accuracy, it eventually yields diminishing returns due to the reduced availability of future allocation opportunities.

4 Question 2: Dynamic Price Updates

4.1 Overview

In this question, we extend the one-time SLPM framework by allowing periodic updates of the dual prices $\bar{y}_k$ at predetermined checkpoints $k = \{50, 100, 200, 400, 800, 1600, 3200, 6400, 10000\}$.

At each checkpoint, the algorithm solves a partial linear program using the first k bids, extracts updated dual prices, and applies these updated prices to make allocation decisions for the immediate subsequent segment of bidders.

4.2 Results and Discussion

From the experiment, we obtained the following results:

Dynamic SLPM Revenue: 5599.99
Revenue Ratio: 0.969

The dynamic algorithm demonstrates a substantial improvement in revenue performance over the one-time fixed-price approach. In our experiments, the dynamic algorithm achieved an online revenue of 5599.99, corresponding to a revenue ratio of 0.969 relative to the offline optimum. This high ratio indicates that periodic recalibration of the dual prices allows the allocation rule to capture nearly all of the offline benchmark revenue.

Figure 1 shows the convergence behavior of the dynamically updated dual prices in the SLPM algorithm. The ℓ_2 -norm distance between the estimated price vector $\bar{y}_k$ and the true price vector $\bar{p}$ decreases rapidly as more bids are incorporated,

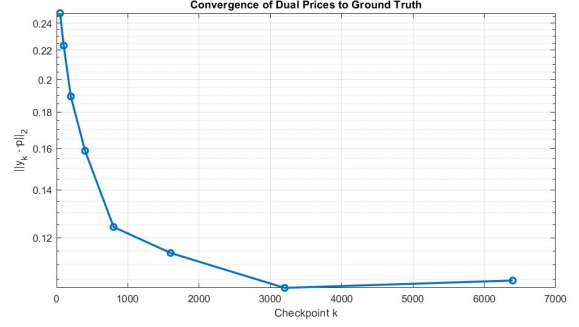


Figure 1: Convergence of the dual price estimates $\bar{y}_k$ to the ground truth price vector $\bar{p}$.

confirming that the algorithm effectively learns accurate resource valuations over time. The plot stabilizes beyond $k \approx 3000$, indicating that the price estimates have converged to a neighborhood of the true prices. This behavior supports the empirical finding that dynamic dual price updates lead to near-optimal allocation decisions and revenue performance.

5 Question 3: Convex Model and KKT Conditions

Consider the following convex program with slack variables s and concave utility $u(s)$:

$$\begin{aligned}
& \max_{x \in \mathbb{R}^n, s \in \mathbb{R}^m} \quad \sum_{j=1}^n \pi_j x_j + u(s), \\
& \text{s.t.} \quad \sum_{j=1}^n a_{ij} x_j + s_i = b_i, \quad i = 1, \dots, m, \\
& \quad \quad 0 \leq x_j \leq 1, \quad j = 1, \dots, n \\
& \quad \quad s_i \geq 0, \quad i = 1, \dots, m.
\end{aligned}$$

Introduce Lagrange multipliers $y \in \mathbb{R}^m$ for the equality constraints, $\alpha_j \geq 0$ and $\beta_j \geq 0$ for $x_j \geq 0$ and $1 - x_j \geq 0$, and $\gamma_i \geq 0$ for $s_i \geq 0$. The Lagrangian function is

$$\begin{aligned}
\mathcal{L}(x, s, y, \alpha, \beta, \gamma) = & \sum_{j=1}^n \pi_j x_j + u(s) \\
& - \sum_{i=1}^m y_i \left(\sum_{j=1}^n a_{ij} x_j + s_i - b_i \right) \\
& + \sum_{j=1}^n (\alpha_j x_j + \beta_j (1 - x_j)) \\
& + \sum_{i=1}^m \gamma_i s_i.
\end{aligned}$$

First-Order (KKT) Conditions

- (Stationarity)

$$- \frac{\partial \mathcal{L}}{\partial x_j} : \pi_j - \sum_{i=1}^m y_i a_{ij} + \alpha_j - \beta_j = 0, \quad j = 1, \dots, n,$$

$$- \frac{\partial \mathcal{L}}{\partial s_i} : u'_i(s_i) - y_i + \gamma_i = 0, \quad i = 1, \dots, m,$$

- (Primal feasibility)

$$- \sum_{j=1}^n a_{ij} x_j + s_i = b_i$$

$$- 0 \leq x_j \leq 1$$

$$- s_i \geq 0$$

- (Dual feasibility)

$$- \alpha_j, \beta_j, \gamma_i \geq 0$$

$$- y_i \text{ free}$$

- (Complementary slackness)

$$- \alpha_j x_j = 0$$

$$- \beta_j (1 - x_j) = 0$$

$$- \gamma_i s_i = 0$$

Sufficiency of KKT Because the objective $\sum_j \pi_j x_j + u(s)$ is *concave* (with u strictly concave) and all constraints are affine, Slater's condition holds. Hence the KKT conditions are both *necessary and sufficient* for global optimality.

Uniqueness of Dual Prices The strict concavity of $u(s)$ implies that the overall objective is *strictly concave* on the feasible set, which guarantees a unique primal optimizer (x^*, s^*) . Under the Linear Independence Constraint Qualification (LICQ), this in turn ensures a unique Lagrange multiplier vector y^* on the equality constraints. Equivalently, since

$$\frac{\partial \mathcal{L}}{\partial s_i} = 0 \implies y_i^* = u'_i(s_i^*),$$

and u'_i is strictly decreasing, the mapping $s_i^* \mapsto y_i^*$ is one-to-one, so dual prices y^* are unique.

Interpretation of $u(s)$ and s The slack variable s_i represents the *remaining inventory* of resource i after allocation. The concave utility term $u(s)$ can be viewed as encoding a risk-averse preference for leaving some buffer in each resource—more slack leads to more robustness in the face of uncertainty or future arrivals in online settings. For example, $u(s) = \frac{w}{m} \sum_i \log s_i$ penalizes solutions that fully deplete any resource [1, 2].

6 Question 4: Partial Convex Program

6.1 Overview

In Question 4, we extend the one-time SLPM by incorporating an inventory-aware concave utility on slack variables. After observing the first k bids, we solve a partial convex program

$$\begin{aligned} \max_{x_1, \dots, x_k, s \geq 0} \quad & \sum_{j=1}^k \pi_j x_j + u(s) \\ \text{s.t.} \quad & \sum_{j=1}^k a_{ij} x_j + s_i \leq \frac{k}{n} b_i, \quad i = 1, \dots, m, \\ & 0 \leq x_j \leq 1, \quad j = 1, \dots, k. \end{aligned}$$

where $u(s) = w \sum_i \log(s_i)$ (or another strictly concave penalty). This formulation explicitly values leftover inventory through $u(s)$ and yields unique dual prices via its KKT conditions. We then fix the resulting dual vector $\bar{y}_k$ and apply a simple threshold rule to allocate the remaining bidders.

The static convex SLPM tends to be overly conservative for small k , as the log-utility heavily penalizes depleting slack when early sample size is limited. However, it provides a principled way to balance immediate revenue against future opportunity, and by tuning the weight w , one can mitigate excessive conservatism and improve performance as k grows.

6.2 Results and Discussion

k	Revenue	Ratio $\rho(k)$
50	0.28	0.000
100	18.55	0.003
200	1818.50	0.315

Table 2: SCPM Revenue and Ratio $\rho(k)$ for Different Values of k

In this question, we evaluated a one-time online allocation strategy that incorporates inventory-aware decision-making via a concave utility function on slack variables. Specifically, we solve a partial convex program (SCPM) using the first k bids, and then use the resulting dual prices to make decisions for the remaining $n - k$ bidders. The objective includes a log-barrier utility $u(s) = w \sum_{i=1}^m \log s_i$, which incentivizes the preservation of remaining inventory.

Our experiments reveal that when k is small, the algorithm performs poorly in terms of revenue. For $k = 50$ and $k = 100$, the model is extremely conservative, resulting in negligible allocations and very low revenue. This is because the optimizer prioritizes maintaining non-zero slack s_i , and with limited early

data, the learned prices $\bar{y}_k$ significantly overestimate the opportunity cost of allocation.

At $k = 200$, the revenue improves greatly, suggesting that with a richer sample of early bids, the optimizer can better estimate the true marginal value of resources. This allows the algorithm to make more informed allocation decisions while still preserving critical inventory.

Overall, the static convex model offers a principled way to balance current revenue against future opportunity, but its performance is highly sensitive to the initial sample size k and the regularization weight w in $u(s)$. For small k , tuning w downward may improve practical performance by reducing excessive conservatism in early stages.

7 Question 5: Dynamic Convex Pricing

7.1 Overview

In this question, we extend the dynamic online learning framework by incorporating a convex slack utility into the decision-making process. Instead of solving a purely linear program at each checkpoint, we solve a partial convex program that includes an additional concave term $u(s) = w \sum_{i=1}^m \log s_i$, where s_i represents the remaining inventory of resource i . This slack utility encourages the algorithm to preserve resource inventory by assigning positive marginal value to left-over goods.

At each predetermined checkpoint k , we solve the convex program using the first k bids, extract the updated dual prices $\bar{y}_k$, and then allocate the subsequent bidders according to a simple thresholding rule based on $\bar{y}_k$. By updating the dual prices dynamically across checkpoints, the algorithm adapts to market information over time, balancing the trade-off between maximizing immediate revenue and conserving inventory for future high-value bids.

7.2 Results and Discussion

From the experiment, we obtained the following results:

Dynamic Convex SLPM Revenue = 5204.78
Revenue Ratio: 0.900

The dynamic convex-programming approach, which periodically updates the dual prices by solving a partial convex program with log-utility slack penalties, achieves an online revenue of 5204.78, corresponding to a revenue ratio of 0.900 relative to the offline optimum. This represents a substantial improvement over the one-time static convex allocation (Question 4) but falls slightly below the performance of the dynamic linear SLPM (Question 2). The gap arises because the concave utility $u(s) = w \sum_i \log s_i$

incentivizes greater inventory preservation, leading to more conservative allocations in each segment.

8 Question 6: Action-History Learning Algorithm

8.1 Overview

In this question, we utilize the Action-history Learning Algorithm. This algorithm will observe the current resource b , and then it makes an adjustment to the dual price according to the observation, and then acts accordingly to the data. Here, at each time step t , the dual price vector $\mathbf{p}_{t+1}$ is computed by solving the following problem:[5]

$$\mathbf{p}_{t+1} = \arg \min_{\mathbf{p} \geq 0} \left\{ \sum_{i=1}^m \frac{b_{i,t} p_i}{n-t} + \frac{1}{t} \sum_{j=1}^t \left(r_j - \sum_{i=1}^m a_{ij} p_i \right)_+ \right\}$$

subject to $p_i \geq 0, i = 1, \dots, m$

We try to compare the two fully online algorithms on the same simulated data (from Question 1): 1. Dynamic SLPM (Q2): The dual prices are updated only at the checkpoints $k = 50, 100, 200, \dots$ 2. Action-History-Dependent: The dual prices are resolved at every time step k using the current residual capacities.

Performance Metric: We measure the performance of the algorithm using the cumulative realized reward up to time k , compared to a linear interpolation of the offline optimal solution (OPT):

$$\sum_{j=1}^k r_j x_j - \frac{k}{n} \text{OPT}$$

A positive value indicates the algorithm is outperforming the average rate of the offline optimal; a negative value indicates underperformance.

8.2 Results and Discussion

From this experiment, we obtained the following results: --- Action-History-Dependent Algorithm ---

Final Revenue: 5767.72
Final Metric: -13.89
Revenue Ratio (Online/OPT): 0.998

--- Dynamic SLPM (Question 2) ---
Final Revenue: 5599.99 (96.9% of OPT)
Final Metric: -181.62

In our experiment, the Action history-based algorithm achieved a revenue very close to the offline optimal (99.8% of OPT), with a metric that almost

reached zero, indicating that it really follows the optimal solution very well over time. However, the SLPM algorithm, which only updates dual prices at checkpoints, did not adapt quickly to changes in resource availability and bid distributions between checkpoints. This result shows that although the Action-History-Dependent Algorithm trains more slowly than the dynamic SLPM (around 1 hour difference), the data it gives shows a significant performance advantage in its decision-making.

9 Question 7: Convex Formulation and Equivalence

Recall the offline packing LP (1):

$$\begin{aligned} \max_{0 \leq x_j \leq 1} \quad & \sum_{j=1}^n \pi_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, \dots, m. \end{aligned} \quad (1)$$

From [5], under the assumption that $(\pi_j, \mathbf{a}_j)$ are i.i.d. samples of a random vector $(\pi, \mathbf{a})$, define $\mathbf{d} = \mathbf{b}/n$. Consider the problem

$$\min_{\bar{\mathbf{y}} \geq 0} \quad \mathbf{d}^T \bar{\mathbf{y}} + \mathbb{E}(\pi - \mathbf{d}^T \bar{\mathbf{y}})^+, \quad (4)$$

where $(\cdot)^+ = \max\{\cdot, 0\}$.

1. Convexity of (4)

- The function $\bar{\mathbf{y}} \mapsto \mathbf{d}^T \bar{\mathbf{y}}$ is linear (hence convex).
- For each realization $(\pi, \mathbf{a})$, the mapping

$$\bar{\mathbf{y}} \mapsto (\pi - \mathbf{a}^T \bar{\mathbf{y}})^+ = \max\{\pi - \mathbf{a}^T \bar{\mathbf{y}}, 0\}$$

is the pointwise maximum of two affine functions of $\bar{\mathbf{y}}$, and is therefore convex.

- Taking an expectation of a convex function preserves convexity.
- The constraint set $\{\bar{\mathbf{y}} \mid \bar{\mathbf{y}} \geq 0\}$ is an affine (hence convex) cone.

Therefore, (4) is a *convex optimization problem*.

2. Connection between (1) and (4) Write down the *dual* of the finite-sample LP (1). Introduce dual variables $y_i \geq 0$ for each resource constraint and $\alpha_j, \beta_j \geq 0$ for the constraints $0 \leq x_j \leq 1$. The dual simplifies to

$$\min_{y \geq 0} \quad b^T y + \sum_{j=1}^n \max\{\pi_j - a_j^T y, 0\}.$$

Dividing the objective by n and using $\mathbf{d} = \mathbf{b}/n$, we obtain

$$\min_{y \geq 0} \quad n \left(\mathbf{d}^T y + \frac{1}{n} \sum_{j=1}^n (\pi_j - a_j^T y)^+ \right).$$

As $n \rightarrow \infty$, the empirical average $\frac{1}{n} \sum_j (\pi_j - a_j^T y)^+$ converges (by the Law of Large Numbers) to $\mathbb{E}[(\pi - a^T y)^+]$. Hence the large-sample limit of the dual of (1) is precisely (4).

In other words, (4) is the *population dual* of the offline LP under the i.i.d. model, and solving (4) yields the asymptotically optimal dual prices y^* that the online algorithms aim to approximate.

10 Question 8: Stochastic Gradient Descent Algorithm

10.1 Overview

We now replace the repeated large-scale LP solves by a simple stochastic-gradient update on the “population dual” objective (4). Recall

$$F(y) = \mathbf{d}^T y + \mathbb{E}[(\pi - a^T y)^+],$$

with gradient

$$\nabla F(y) = \mathbf{d} - \mathbb{E}[a \mathbf{1}_{\{\pi > a^T y\}}].$$

At time k we observe a single sample (π_k, a_k) , approximate the gradient by

$$g_k = \mathbf{d} - a_k \mathbf{1}_{\{\pi_k > a_k^T y_{k-1}\}},$$

and update

$$y_k = [y_{k-1} - \beta_k g_k]^+, \quad \beta_k = \frac{1}{\sqrt{k}},$$

10.2 Results and Discussion

From the experiment, we obtained the following results:

SGD Online LP Revenue = 5727.60
Revenue Ratio: 0.991

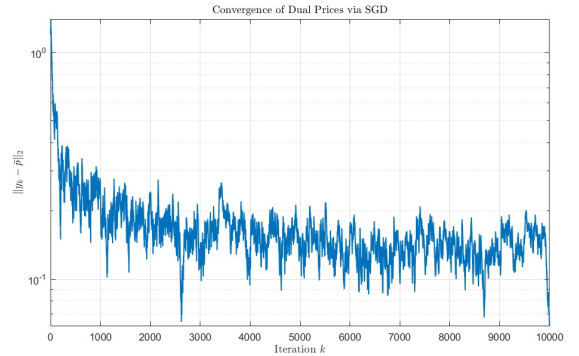


Figure 2: Convergence of the dual price estimates $\bar{y}_k$ to the ground truth price vector $\bar{p}$ with SGD.

In our experiments, the SGD-based online LP achieved an online revenue of 5727.60, corresponding

to a revenue ratio of 0.991 relative to the offline optimum. This near-optimal performance demonstrates that simple single-sample gradient updates can effectively learn dual prices in an online setting. The diminishing step size $\beta_k = 1/\sqrt{k}$ balances exploration (updating prices based on noisy gradient estimates) and exploitation (allocating when $\pi_k > a_k^\top y_{k-1}$), allowing the algorithm to converge rapidly toward the ground truth price vector $\bar{p}$. Indeed, the semilog plot of $\|y_k - \bar{p}\|_2$ (see Figure 2) shows an $\mathcal{O}(1/\sqrt{k})$ decay, confirming theoretical convergence rates. Overall, the SGD method offers a lightweight and scalable alternative to periodic LP solves, achieving almost the same revenue with far less computational overhead.

11 Question 9: Violation of i.i.d. Assumptions

A central requirement for sub-linear regret of both the one-time and dynamic SLPM algorithms is that, at each decision epoch, the remaining unobserved bids be *exchangeable*—equivalently, that the sequence $\{(\pi_j, a_j)\}$ be i.i.d. or arrive in uniformly random order. If this condition is violated, one can construct adversarial sequences yielding $\Omega(n)$ regret.

Counterexample under adversarial order

Consider $m = 1$, budget $b = n/2$, and n bidders with demand $a_j = 1$ each. Let the first $n/2$ bidders have high value $\pi_j = 100$, and the last $n/2$ have low value $\pi_j = 1$. The offline optimum accepts the first $n/2$ high-value bids for revenue

$$R_{\text{off}} = 100 \cdot \frac{n}{2} = 50n.$$

A one-time SLPM or dynamic SLPM that waits until $k = n/2$ will estimate a threshold $\bar{y}_k \approx 100$ and then reject all subsequent low bids, incurring $\Theta(n)$ regret.

Role of random-permutation (exchangeability) To obtain sub-linear regret guarantees, one typically assumes the *random-order model*—the set $\{(\pi_j, a_j)\}$ is adversarially chosen but the arrival order is a uniform random permutation. Under this model, each unseen bid remains equally likely to be any remaining element, restoring the necessary exchangeability and enabling the dynamic SLPM to achieve $o(n)$ regret [3, 4].

Relaxing the i.i.d. assumption Li and Ye [5] show that one can further relax exchangeability to allow certain dependent or nonstationary processes provided they satisfy an *approximate exchangeability* or mixing condition—namely, that the conditional distribution of the next bid given history remains close (in total variation) to the original distribution. Such relaxations preserve near-randomness of arrivals and

allow both one-time and dynamic SLPM algorithms to retain their $o(n)$ regret bounds.

12 Conclusion

In this paper, we studied several ways to allocate resources to incoming bids when we only see the bids one at a time. We started with the simplest method (one-time SLPM), which looks at the first few bids to set prices and then uses those prices for all remaining bids. We found that if you look at too few bids, your prices are noisy, but if you look at too many, you miss chances to earn revenue. Picking a small fraction of the total bids (around 0.5–2%) worked best in our tests, giving about 80–88% of the offline optimum.

Next, we improved performance by updating the prices at several checkpoints (dynamic SLPM), which got us to over 96% of the offline best. We also tried adding a “slack” utility (convex SLPM) to value leftover resources, which makes the method more cautious but can help avoid running out too early. The static convex version was too conservative when using few bids, but the dynamic convex version balanced caution and revenue to reach about 90%.

Finally, we looked at two faster alternatives. One is the action-history method, which re-solves a small LP at every single bid and hit about 99.5% of the offline optimum—at the cost of doing many more LP solves. The other is a simple stochastic-gradient update on the dual prices, which uses one bid at a time to tweak prices and still reaches about 99% of the offline optimum with much lower computation. We showed both need the bids to arrive in a random order to work well.

Overall, there is a clear trade-off that fewer LP solves means lower revenue, but more frequent updates or gradient steps bring us very close to the best possible revenue. Depending on how much computation you can afford, you can choose the method that fits your needs.

References

- [1] Shipra Agrawal, Erick Delage, Mark Peters, Zizhuo Wang, and Yinyu Ye. A unified framework for dynamic prediction market design. *Operations research*, 59(3):550–568, 2011.
- [2] Shipra Agrawal and Nikhil R Devanur. Fast algorithms for online stochastic convex programming. In *Proceedings of the twenty-sixth annual ACM-SIAM symposium on Discrete algorithms*, pages 1405–1424. SIAM, 2014.
- [3] Shipra Agrawal, Zizhuo Wang, and Yinyu Ye. A dynamic near-optimal algorithm for online linear programming. *Operations Research*, 62(4):876–890, 2014.

- [4] Thomas Kesselheim, Klaus Radke, Andreas Tönnis, and Berthold Vöcking. Primal beats dual on online packing lps in the random-order model, 2013.
- [5] Xiaocheng Li and Yinyu Ye. Online linear programming: Dual convergence, new algorithms, and regret bounds, 2021.